ATIVIDADE SOBRE COESÃO RELATÓRIO TÉCNICO

Projeto em C com módulos de coesão funcional e sequencial

1. Objetivo da atividade

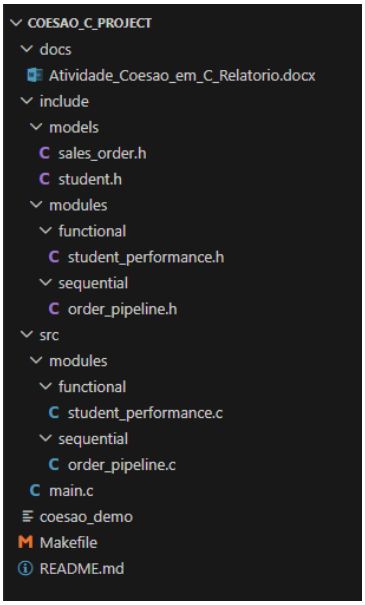
Este projeto foi desenvolvido para mostrar, na prática, **dois tipos clássicos de coesão de módulos em linguagem C**: a coesão funcional e a coesão sequencial. A proposta foi organizar o código em arquivos separados, com responsabilidades claras, nomes autoexplicativos e fluxo simples de acompanhar.

A solução foi estruturada para atender quatro metas principais:

- demonstrar os conceitos de forma objetiva e correta;
- produzir um código limpo, comentado e fácil de manter;
- facilitar a explicação oral da atividade;
- entregar um projeto que compile e execute sem ajustes extras.

2. Organização do projeto

A estrutura foi dividida em *models* (estruturas de dados), *modules* (regras de negócio) e um *main* enxuto, responsável apenas por demonstrar os módulos.



3. Módulo com coesão funcional

Arquivo principal: student_performance.c

A coesão funcional ocorre quando todas as funções de um módulo existem para cumprir **um único objetivo central**. Neste caso, o objetivo é avaliar o desempenho de um aluno.

Função	Papel no módulo funcional
calculate_average	Calcula a média final do aluno.
find_highest_score	Obtém a maior nota.
find_lowest_score	Obtém a menor nota.
has_minimum_attendance	Valida a frequência mínima.
get_student_status	Define a situação final do aluno.
print_student_report	Gera a saída consolidada do relatório.

Interpretação acadêmica: mesmo havendo várias funções, todas estão ligadas à mesma responsabilidade. Isso caracteriza coesão funcional.

4. Módulo com coesão sequencial

Arquivo principal: order_pipeline.c

A coesão sequencial acontece quando o módulo executa uma **cadeia de etapas**, em que a saída de uma fase serve de base para a próxima.

1. initialize_processed_order → copia e prepara os dados iniciais;
2. calculate_gross_total → calcula o total bruto;
3. calculate_discount → usa o total bruto para obter o desconto;
4. calculate_net_total → usa bruto e desconto para obter o total líquido;
5. define_order_status → usa o total líquido para concluir o status final.

Interpretação acadêmica: o módulo funciona como um pipeline. A ordem das etapas é importante, e cada etapa depende do resultado produzido antes.

5. Como o main.c foi mantido limpo

O arquivo main.c não concentra regra de negócio. Ele apenas chama duas funções demonstrativas:

- `demonstrate_functional_cohesion();`
- `demonstrate_sequential_cohesion();`

Essa decisão segue Clean Code, porque reduz acoplamento e mantém o ponto de entrada simples.

6. Trechos essenciais do código

Abaixo estão os trechos mais importantes.

Trecho 1 - Cabeçalho do módulo funcional

student_performance.h

include > modules > functional > student_performance.h > ...

```c
#ifndef STUDENT_PERFORMANCE_H
#define STUDENT_PERFORMANCE_H

#include "models/student.h"

float calculate_average(const Student *student);
float find_highest_score(const Student *student);
float find_lowest_score(const Student *student);
int has_minimum_attendance(const Student *student);
const char *get_student_status(const Student *student);
void print_student_report(const Student *student);

#endif
```

Trecho 2 - Decisão da situação do aluno

```c

    return lowest_score;
}

int has_minimum_attendance(const Student *student) {
    /* Regra simples de frequência mínima. */
    if (is_student_pointer_invalid(student)) {
        return 0;
    }

    return student->attendance_rate >= 75.0f;
}

const char *get_student_status(const Student *student) {
    /* Consolida as regras acadêmicas de aprovação. */
    if (is_student_pointer_invalid(student)) {
        return "Dados invalidos";
    }

    const float average = calculate_average(student);
    const int attendance_is_valid = has_minimum_attendance(student);

    if (!attendance_is_valid) {
        return "Reprovado por falta";
    }

    if (average >= 7.0f) {
        return "Aprovado";
    }

    if (average >= 5.0f) {
        return "Recuperacao";
    }

    return "Reprovado por nota";
}

```

Trecho 3 - Pipeline sequencial principal

```c

    strcpy(processed_order->status, "Invalido");
}

void process_order(const SalesOrder *input_order, ProcessedOrder *processed_order) {
    /* Orquestra todas as etapas em ordem sequencial. */
    if (input_order == NULL || processed_order == NULL) {
        return;
    }

    initialize_processed_order(input_order, processed_order);
    calculate_gross_total(processed_order);
    calculate_discount(processed_order, input_order->discount_percentage);
    calculate_net_total(processed_order);
    define_order_status(processed_order);
}

```

Trecho 4 - Uso dos módulos no main

```c

/* Demonstra o modulo funcional com um exemplo de aluno. */
static void demonstrate_functional_cohesion(void) {
    const Student student = {
        .name = "Ana Carolina",
        .exam_scores = {8.5f, 7.0f, 9.0f, 6.5f},
        .attendance_rate = 92.0f
    };

    print_student_report(&student);
}

/* Demonstra o modulo sequencial com um exemplo de pedido. */
static void demonstrate_sequential_cohesion(void) {
    const SalesOrder input_order = {
        .customer_name = "Carlos Henrique",
        .product_name = "Notebook para estudo",
        .quantity = 2,
        .unit_price = 350.0f,
        .discount_percentage = 10.0f
    };

    ProcessedOrder processed_order;
    process_order(&input_order, &processed_order);
    print_processed_order(&processed_order);
}

/* Ponto de entrada: apenas organiza a demonstracao dos modulos. */
int main(void) {
    printf("Projeto demonstrativo sobre coesao em C\n");
    printf("- Modulo 1: coesao funcional\n");
    printf("- Modulo 2: coesao sequencial\n");

    demonstrate_functional_cohesion();
    demonstrate_sequential_cohesion();

    return 0;
}

```

7. Boas práticas de Clean Code adotadas

- nomes de funções claros e orientados à intenção;
- funções pequenas e com responsabilidade única;
- separação entre definição (.h) e implementação (.c);
- uso de structs para representar entidades do domínio;
- main desacoplado das regras de negócio;
- comentários usados para apoiar entendimento, sem poluir o código.

8. Conclusão

O projeto atende integralmente à proposta da atividade. Ele demonstra os dois tipos de coesão solicitados com exemplos concretos, legíveis e defensáveis em sala. Além disso, a estrutura modular facilita tanto a correção pelo professor quanto a explicação pelos alunos.

saída final do terminal após executar o programa com sucesso.

Comando de compilação utilizado no Windows (PowerShell): gcc -I./include ./src/main.c ./src/modules/functional/student_performance.c ./src/modules/sequential/order_pipeline.c -o ./bin/coesao.exe

```
PROBLEMS 9    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\user\Documents\Puc\10 periodo\coesao_c_project> mkdir bin -ErrorAction SilentlyContinue
>> gcc -I./include ./src/main.c ./src/modules/functional/student_performance.c ./src/modules/sequential/order_pipeline.c -o ./bin/
>> .\bin\coesao.exe
Projeto demonstrativo sobre coesao em C
- Modulo 1: coesao funcional
- Modulo 2: coesao sequencial

=== RELATORIO DE DESEMPENHO DO ALUNO ===
Nome: Ana Carolina
Notas: 8.5 | 7.0 | 9.0 | 6.5
Media: 7.75
Maior nota: 9.0
Menor nota: 6.5
Frequencia: 92.0%
Situacao final: Aprovado

=== RESULTADO DO PROCESSAMENTO SEQUENCIAL ===
Cliente: Carlos Henrique
Produto: Notebook para estudo
Quantidade: 2
Preco unitario: R$ 350.00
Total bruto: R$ 700.00
Desconto: R$ 70.00
Total liquido: R$ 630.00
Status: Aprovado com prioridade
PS C:\Users\user\Documents\Puc\10 periodo\coesao_c_project>
```